

# Software Requirements Specification

*for*

## Ekalavyaa

*A Self-Learning Platform with Section-Locking Quizzes & Gurukul Coin Economy*

---

PREPARED BY

Kirankumar

PRODUCT OWNER

AI Medha Labs Private Limited

DOCUMENT STANDARD

IEEE Std 830-1998 (Recommended  
Practice for SRS)

VERSION & DATE

Version 1.0 — June 2026

# Quick Read — For Interns Joining the Project

*If you're new to the team, read this one page first. Everything that follows is the formal spec — useful as a reference, but not required reading on day one. This page tells you what Ekalavyaa actually is and what makes it different.*

## What is Ekalavyaa, in 30 seconds?

Ekalavyaa is a web learning platform. A student signs up, picks a course, watches YouTube lectures inside our site, takes a multiple-choice quiz after each section, and unlocks the next section only if they score at least 70%. As they learn, they earn "Gurukul coins" — a fun progress reward, not real money. They get a certificate once every quiz in a course is passed. Think of it as an opinionated Udemy built around *self-driven learning* (named after Ekalavya, the self-taught archer of the Mahabharata).

## What makes Ekalavyaa different from a regular Udemy clone?

Two features are the "novelty" — the reason this project exists:

**1. Upload-your-own-syllabus → get-a-custom-course.** A student uploads a PDF of their college syllabus (or pastes the text in). Our backend sends that text to Google Gemini, asks it to break the syllabus into 6 sections of 5 lectures each, searches YouTube for a matching video per lecture, and saves it all as a brand-new course — auto-enrolling the student. *Nothing else in this space does that.*

**2. Dronacharya — the in-platform AI tutor.** A friendly chatbot named after Arjuna's guru. We feed it the live catalogue on every message, so it can answer "which course teaches Python?" or "how do I unlock the next section?" without making things up. Powered by Gemini, with a rule-based fallback if the API is down.

— continued on the next page —

# Quick Read — continued

## A normal student's journey, step by step

- 1** Open `ekalavyaa.onrender.com` → **Sign up** → enter name / email / password.
- 2** Click **Browse courses** → pick "Python From Zero to Hero" → **Enroll**.
- 3** Watch lecture 1 (a YouTube video embedded in our page) → click **Mark complete** → earn **5 coins**.
- 4** After the last lecture of Section 1, take the quiz. Score 70%+ → earn **20 coins** → Section 2 unlocks.
- 5** Pass every quiz → earn a **100-coin bonus** + a certificate of completion.
- 6** Want a custom course? Go to **Upload Syllabus**, upload a PDF — the system builds one on the fly.

## The stack — what an intern should know

- **Backend:** Python 3.12 + Flask 3.0, server-rendered Jinja2 templates. No React, no SPA.
- **Database:** SQLite locally, PostgreSQL in production — same Python code ( `database.py` auto-switches on `DATABASE_URL` ).
- **AI layer:** Google Gemini via `google-generativeai` . Heuristic fallback when `GEMINI_API_KEY` isn't set — nothing breaks.
- **YouTube:** No API key. We scrape the search results page and pull the first video ID.
- **Hosting:** Render.com (free). Auto-deploys on every `git push` to `main` . Gunicorn serves the app.
- **Auth:** Server-side sessions; passwords hashed with Werkzeug PBKDF2.

## What to read next

Building a feature? Go to **Section 4** (Functional Requirements). For the novelty features, see **Sections 4.8 and 4.9**. For deployment, read **Section 6**. The Table of Contents on the next page is your map.

# Table of Contents

---

A *Quick Read for Interns* appears on pages 2–3 — a plain-language overview of the whole platform. The numbered sections below begin on page 5.

## Introduction5

|     |                                           |   |
|-----|-------------------------------------------|---|
| 0.1 | Purpose                                   | 5 |
| 0.2 | Document Conventions                      | 5 |
| 0.3 | Intended Audience and Reading Suggestions | 5 |
| 0.4 | Project Scope                             | 6 |
| 0.5 | References                                | 6 |

## Overall Description7

|     |                                       |   |
|-----|---------------------------------------|---|
| 0.1 | Product Perspective                   | 7 |
| 0.2 | Product Features                      | 7 |
| 0.3 | User Classes and Characteristics      | 8 |
| 0.4 | Operating Environment                 | 8 |
| 0.5 | Design and Implementation Constraints | 9 |
| 0.6 | Assumptions and Dependencies          | 9 |

## External Interface Requirements10

|     |                          |    |
|-----|--------------------------|----|
| 0.1 | User Interfaces          | 10 |
| 0.2 | Hardware Interfaces      | 10 |
| 0.3 | Software Interfaces      | 10 |
| 0.4 | Communication Interfaces | 11 |

## System Features (Functional Requirements)12

|     |                                               |    |
|-----|-----------------------------------------------|----|
| 0.1 | User Registration & Authentication            | 12 |
| 0.2 | Course Catalog Browsing                       | 13 |
| 0.3 | Course Enrollment                             | 13 |
| 0.4 | Lecture Viewing & Completion                  | 14 |
| 0.5 | Quizzes & Section Unlocking                   | 14 |
| 0.6 | Gurukul Coin Economy                          | 15 |
| 0.7 | "My Learning" Dashboard                       | 15 |
| 0.8 | Syllabus → Custom Course Generation (Novelty) | 16 |

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements of **Ekalavyaa**, a web-based self-learning platform inspired by commercial offerings such as Udemy and Coursera but rooted in the philosophy of self-taught mastery (the legend of Ekalavya from the Mahabharata). The platform allows registered users to enroll in courses, watch embedded YouTube lectures, attempt section-end quizzes, and progress through a course only after demonstrating competence — with a small in-platform "Gurukul coin" economy rewarding consistent engagement.

This document is intended to serve as the contractual reference between the developer (Kirankumar), the product owner (**AI Medha Labs Private Limited**), and any external reviewer, and as the authoritative guide for design, implementation, testing, and acceptance of the system.

## 1.2 Document Conventions

The following conventions are used throughout this document:

- Functional requirements are identified as `FR-XX` (e.g., `FR-01`).
- Non-functional requirements are identified as `NFR-XX`.
- Priority levels follow RFC 2119 conventions: **MUST** (essential), **SHOULD** (highly recommended), **MAY** (optional).
- References to code identifiers, file paths, and HTTP routes appear in `monospace`.
- External products and standards are presented in *italics* on first mention.

## 1.3 Intended Audience and Reading Suggestions

| READER                        | SECTIONS OF PRIMARY INTEREST                                                              |
|-------------------------------|-------------------------------------------------------------------------------------------|
| Project Supervisor / Examiner | Sections 1, 2, 4, 5 — for an end-to-end view of scope, behaviour, and quality attributes. |
| Developer / Maintainer        | All sections, especially 3, 4, 6 — interface, feature, and database requirements.         |
| Quality Assurance / Tester    | Sections 4, 5 — for testable functional and performance criteria.                         |
| End User (Learner)            | Sections 2.2, 2.3, 4 — to understand the offered features and user roles.                 |
| Future Contributor            | Sections 2.5, 6 — for constraints and deployment context before extending the system.     |

## 1.4 Project Scope

Ekalavyaa is a single-tenant, web-delivered self-learning platform. The first release (v1.0) targets a single category of learner — an individual studying independently, without instructor moderation or live cohorts. The system shall:

- Allow a visitor to browse a catalogue of pre-seeded courses without authentication.
- Allow a visitor to register a personal account using email and password, or a one-click mock Google sign-in for demonstration purposes.
- Allow a registered learner to enroll in any listed course.
- Allow an enrolled learner to consume video lectures (embedded from YouTube) section by section.
- Require successful completion of a section-end multiple-choice quiz ( $\geq 70\%$  by default) to unlock the next section.
- Award a virtual currency ("Gurukul coins") for each completed lecture, each passed quiz, and each course completion, and maintain an audit ledger of all coin events.
- Provide a personal dashboard summarising enrolled courses and progress.
- **Principal novelty.** Accept a learner-supplied syllabus (PDF, TXT, MD, or pasted text) and synthesise a customised course from it — using a Large Language Model (Google Gemini) to structure the topics into sections and lectures, sourcing video resources by YouTube search, generating quizzes from the same topics, and persisting the result as a real course that the uploading learner is automatically enrolled in.
- **Principal novelty.** Provide an in-platform conversational tutor (the "Dronacharya" chatbot) that is aware of the live course catalogue and answers learner questions about courses, enrollment, certification, navigation, and account state.

The following are **explicitly out of scope** for v1.0: instructor account creation, course authoring UI, video hosting (YouTube is used as a delegated provider), real Google OAuth integration, payment processing, certificate generation, discussion forums, mobile native applications, and live cohorts.

## 1.5 References

- IEEE Std 830-1998 — *Recommended Practice for Software Requirements Specifications*.
- RFC 2119 — *Key words for use in RFCs to Indicate Requirement Levels*.
- Flask Documentation — <https://flask.palletsprojects.com/>
- PostgreSQL 16 Reference Manual — <https://www.postgresql.org/docs/16/>
- SQLite Documentation — <https://www.sqlite.org/docs.html>
- Render Web Service Documentation — <https://render.com/docs/web-services>
- OWASP Top 10 (2021) — for security requirements baseline.

## 2. Overall Description

---

### 2.1 Product Perspective

Ekalavyaa is a self-contained server-rendered web application. It is not a component of a larger product family. Its closest commercial analogues are Udemy and Coursera; however, Ekalavyaa is deliberately simpler and is designed for individual self-study rather than instructor-led delivery. The system relies on two delegated external services: *YouTube* as the video-content provider (lectures are embedded via standard YouTube iframe URLs) and a Postgres-compatible database service (used in production deployment).

From the user's perspective, the system presents a public landing page, a course catalogue, authentication pages, a personal "My Learning" dashboard, course-detail pages with embedded lectures, and quiz pages. From the deployment perspective, the system is a single Flask process served by Gunicorn behind Render's HTTPS load balancer.

### 2.2 Product Features

The principal features of the platform are summarised below. Each is elaborated in Section 4.

- **F1. Account management** — email/password registration, login, logout, and a demonstration Google sign-in.
- **F2. Course discovery** — public catalogue and individual course detail pages.
- **F3. Enrollment** — registered users may enroll in any listed course at no cost.
- **F4. Lecture playback** — embedded YouTube videos with a "Mark complete" action.
- **F5. Quizzes** — multiple-choice quizzes at the end of each section, with a passing threshold (default 70%).
- **F6. Section locking** — subsequent sections are visible but inaccessible until the previous section's quiz has been passed.
- **F7. Gurukul coin economy** — virtual currency awarded for lecture completion (5 coins), quiz pass (20 coins), and full course completion (100 coins).
- **F8. Progress dashboard** — per-course completion percentage and aggregate coin balance shown on the personal dashboard.
- **F9. Syllabus-driven course authoring (novelty)**. The platform's distinguishing feature: a learner uploads a syllabus and the system synthesises a complete custom course, including sections, lectures (with sourced YouTube videos), and section-end quizzes.
- **F10. Dronacharya conversational tutor (novelty)**. An LLM-powered chatbot grounded in the live course catalogue answers learner questions in a warm, teacherly tone.

## 2.3 User Classes and Characteristics

| CLASS              | DESCRIPTION                                                                                                           | FREQUENCY OF USE                        | TECHNICAL SKILL                                 |
|--------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------|-------------------------------------------------|
| Visitor (Guest)    | A non-authenticated user browsing the public catalogue.                                                               | Occasional — pre-conversion only.       | Basic web browsing.                             |
| Registered Learner | The primary user. Has an account, enrolls in courses, consumes lectures, attempts quizzes, accumulates Gurukul coins. | Daily to weekly during active learning. | Basic web browsing; familiar with online video. |
| System Operator    | The developer or maintainer. Triggers re-seeds, monitors Render logs, manages the database connection string.         | As required.                            | Intermediate (CLI, Git, Render dashboard).      |

## 2.4 Operating Environment

The system shall operate in the following environment:

- **Server runtime:** Python 3.12 with Flask 3.0 served by Gunicorn 21 (2 workers, 2 threads, 60-second timeout).
- **Hosting provider:** *Render.com* Web Service (free or paid plan).
- **Storage:** SQLite 3 (development) or PostgreSQL 12+ (production). The connection driver is selected automatically at process start based on the presence and prefix of the `DATABASE_URL` environment variable.
- **Client device:** Any modern desktop or mobile web browser with JavaScript enabled (Chrome 90+, Firefox 90+, Safari 15+, Edge 90+).
- **Network:** The client requires an internet connection capable of streaming standard-definition YouTube video ( $\geq 1$  Mbps recommended).



## 2.5 Design and Implementation Constraints

- The system shall be implemented in Python using the Flask framework. Alternative backend stacks are out of scope.
- All HTML shall be rendered server-side using Jinja2 templates; no single-page application framework shall be introduced in v1.0.
- Stateless HTTP sessions shall be implemented using Flask's signed-cookie session interface backed by `Flask-Session` in filesystem mode for development.
- Passwords shall be hashed using `werkzeug.security.generate_password_hash` (PBKDF2-SHA256 by default). Plain-text password storage is prohibited.
- The database access layer shall expose an identical API for SQLite and Postgres backends; calling code shall not branch on the backend type.
- The system shall not depend on any paid third-party API as a hard requirement; demonstrations shall remain operable on Render's free tier.
- All third-party dependencies shall be pinned to specific versions in `requirements.txt` to ensure reproducibility.

## 2.6 User Documentation

The release shall be accompanied by the following user-facing documentation:

- A `README.md` at the repository root, summarising the platform and giving local run instructions.
- A `DEPLOY.md` giving step-by-step deployment instructions for Render.
- This SRS document, version-controlled alongside the source.

## 2.7 Assumptions and Dependencies

- The YouTube videos referenced by seeded lectures are assumed to remain publicly viewable and embeddable for the lifetime of the deployment.
- The Render free-tier service is assumed to be available; if a paid tier is required, the deployment configuration in `render.yaml` shall be updated accordingly.
- Email delivery is not required by v1.0; password reset and notification flows are deferred to a later release.
- The platform assumes a single seeded administrator/owner. Multi-admin role management is out of scope.

### 3. External Interface Requirements

#### 3.1 User Interfaces

The user interface shall be delivered as responsive HTML pages styled with a single global stylesheet ( `static/css/style.css` ). The visual language draws on contemporary learning platforms: a sticky top navigation bar with brand mark, search field, and sign-in actions; cream/white content areas; a Udemy-inspired purple accent ( `#5624D0` ) for primary call-to-action buttons; and a clean serif/sans-serif typographic pairing for readability.

The following screens shall be provided in v1.0:

1. Landing page ( `/` ) — hero section and featured courses.
2. Sign-up page ( `/signup` ) — registration form.
3. Login page ( `/login` ) — credentials form and mock-Google button.
4. Course catalogue ( `/courses` ) — list of all courses.
5. Course detail ( `/course/<id>` ) — curriculum with section accordion and "Enroll" action.
6. Lecture viewer ( `/lecture/<id>` ) — embedded video player and "Mark complete" action.
7. Quiz ( `/quiz/<id>` ) — multiple-choice form.
8. Quiz result ( `/quiz/<id>/result` ) — score, strong/weak topic breakdown, coin award.
9. "My Learning" dashboard ( `/my-learning` ) — enrolled courses with progress bars and coin balance.
10. 404 page — for any unmatched route.

#### 3.2 Hardware Interfaces

No special hardware interfaces are required. The client device must be capable of running a modern web browser and decoding standard H.264/AAC video as delivered by the YouTube embed.

#### 3.3 Software Interfaces

| INTERFACE                    | DIRECTION                    | DESCRIPTION                                                                                                                                                                                                                   |
|------------------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YouTube IFrame Player        | Outbound (client-side embed) | Lectures are delivered by embedding a YouTube video using the standard <code>https://www.youtube.com/embed/&lt;id&gt;</code> URL pattern. No API key is required for unrestricted public embedding.                           |
| YouTube Search (HTML scrape) | Outbound (server-side)       | The syllabus-to-course pipeline issues a search request to <code>https://www.youtube.com/results</code> per generated lecture title and parses the first matching video identifier from the response. No API key is required. |
| Google Gemini API            | Outbound (server-side)       | Used by both the syllabus pipeline and the Dronacharya chatbot. Activated when <code>GEMINI_API_KEY</code> is configured; model name is configurable via <code>GEMINI_MODEL</code> (default <code>gemini-2.0-flash</code>     |

### 3.4 Communication Interfaces

- All client/server communication shall use HTTPS in production, terminated at the Render load balancer.
- The application speaks HTTP/1.1 between the load balancer and Unicorn.
- The database connection from the application to PostgreSQL shall use TLS ( `sslmode=require` ).
- The HTML pages shall use standard form submissions; no custom protocols or WebSockets are required in v1.0.
- Session cookies shall be marked `HttpOnly` ; in production deployments the `Secure` flag shall additionally be set.

*Note: A subset of the YouTube IFrame Player exchanges telemetry with Google servers. This is a property of the embed and is documented in YouTube's terms of service; it is not under the control of Ekalavyaa.*

#### 3.4.1 Route Map (Reference)

| METHOD    | PATH                   | AUTHENTICATION | PURPOSE                  |
|-----------|------------------------|----------------|--------------------------|
| GET       | /                      | Public         | Landing page.            |
| GET, POST | /signup                | Public         | User registration.       |
| GET, POST | /login                 | Public         | User authentication.     |
| GET       | /auth/google           | Public         | Mock Google sign-in.     |
| GET       | /logout                | Authenticated  | Session destruction.     |
| GET       | /courses               | Public         | Course catalogue.        |
| GET       | /course/<id>           | Public         | Course detail.           |
| POST      | /enroll/<course_id>    | Authenticated  | Enroll the current user. |
| GET       | /lecture/<id>          | Authenticated  | Watch a lecture.         |
| POST      | /lecture/<id>/complete | Authenticated  | Mark a lecture complete. |
| GET, POST | /quiz/<id>             | Authenticated  | Attempt a quiz.          |
| GET       | /my-learning           | Authenticated  | Personal dashboard.      |

## 4. System Features (Functional Requirements)

### 4.1 User Registration & Authentication

The system shall manage the lifecycle of user accounts: registration, authentication, session continuation, and logout.

**FR-01** **MUST**

#### Account Registration

The system **shall** permit a visitor to register a new account by submitting a valid email address, a display name, and a password of length  $\geq 6$  characters. The password shall be hashed using a one-way hash function before storage. On successful registration the user shall be automatically signed in and redirected to the "My Learning" dashboard.

**FR-02** **MUST**

#### Email Uniqueness

The system **shall** reject any registration request that supplies an email address already associated with an existing account. The user shall be informed of the conflict via a flash message and the form shall be re-rendered with previously entered values preserved (except the password).

**FR-03** **MUST**

#### Password Authentication

The system **shall** authenticate a user by comparing the supplied password against the stored hash using a constant-time comparison. On successful authentication the system shall establish a server-side session keyed by the user identifier.

**FR-04** **SHOULD**

#### Demonstration Google Sign-in

The system **should** provide a one-click mock Google sign-in for demonstration purposes only. This shall create or sign into a fixed demo account ( `demo.google@ekalavyaa.com` ) without contacting Google's OAuth endpoints. The mock nature shall be disclosed in any user-facing flash message.

**FR-05** **MUST**

#### Session Termination

The system **shall** support logout by clearing the user session and redirecting to the landing page. Subsequent requests to authenticated routes shall redirect the user to `/login`.

## 4.2 Course Catalog Browsing

**FR-06** **MUST**

### Public Catalogue

The system **shall** display all published courses to any visitor at `/courses` without requiring authentication. Each list item shall display the course title, subtitle, instructor, rating, and primary call-to-action.

**FR-07** **MUST**

### Course Detail

The system **shall** render a per-course detail page at `/course/<id>` showing the full curriculum: ordered sections, each containing ordered lectures (with duration) and a section-end quiz indicator.

## 4.3 Course Enrollment

**FR-08** **MUST**

### Enroll in a Course

The system **shall** permit an authenticated user to enroll in any listed course at no cost. Enrollment shall be idempotent: a duplicate enrollment request for the same (user, course) pair shall succeed without creating a duplicate record or surfacing an error.

**FR-09** **MUST**

### Enrollment Gating

Only an enrolled user **shall** be able to access the lecture viewer or quiz pages for a given course. An unenrolled user attempting either shall be redirected to the course detail page with an appropriate flash message.

## 4.4 Lecture Viewing & Completion

**FR-10** **MUST**

### Lecture Embed

The system **shall** render each lecture as an embedded YouTube video using the URL pattern `https://www.youtube.com/embed/<youtube_id>`. The embed shall be responsive and shall fill the available content width on the lecture page.

**FR-11** **MUST**

### Mark Complete

The system **shall** expose a "Mark complete" action on the lecture viewer. On invocation, the system shall record a `lecture_progress` row for the (user, lecture) pair, idempotently, with a UTC timestamp.

**FR-12** **SHOULD**

### Auto-advance

The system **should** automatically navigate the user to the next lecture in the same section upon completion, or to the section-end quiz if the current lecture is the last in the section.

## 4.5 Quizzes & Section Unlocking

**FR-13** **MUST**

### Quiz Presentation

The system **shall** display each section-end quiz as a sequence of multiple-choice questions. Each question shall present four answer options exactly one of which is correct. The quiz shall be submitted as a single form.

**FR-14** **MUST**

### Quiz Scoring & Pass Threshold

The system **shall** compute the score as the percentage of correctly answered questions, rounded to the nearest integer. A score greater than or equal to the quiz's configured `passing_score` (default 70) shall be classified as a pass.

**FR-15** **MUST**

### Section Locking

The system **shall** render the first section of each enrolled course as unlocked. A subsequent section shall be unlocked only after the user has passed the previous section's quiz at least once.

## 4.6 Gurukul Coin Economy

**FR-17** **MUST**

### Coin Award — Lecture

The system **shall** award 5 coins to the user the first time a given lecture is marked complete. Subsequent completions of the same lecture shall not award coins (idempotent via a unique constraint on the ledger).

**FR-18** **MUST**

### Coin Award — Quiz Pass

The system **shall** award 20 coins to the user the first time a given quiz is passed. Subsequent passes (e.g., higher scores on retry) shall not award additional coins.

**FR-19** **MUST**

### Coin Award — Course Completion

The system **shall** award a one-time bonus of 100 coins when the user has passed every quiz in a course at least once.

**FR-20** **MUST**

### Coin Ledger

The system **shall** persist every coin event in a `coin_ledger` table with the user, amount, reason, optional reference type and id, and a UTC timestamp. The ledger shall enforce uniqueness on (user\_id, reason, ref\_type, ref\_id) to guarantee idempotency.

**FR-21** **SHOULD**

### Coin Balance Display

The system **should** render the current coin balance in the global header on every authenticated page, sourced from the `users.coins` column.

## 4.7 "My Learning" Dashboard

**FR-22** **MUST**

### Enrolled Courses

The system **shall** list every course the authenticated user is enrolled in, ordered by most-recent enrollment first. For each course the system shall display the title, thumbnail, instructor, and percentage progress (computed as completed lectures divided by total lectures, rounded to nearest integer).

## 4.8 Syllabus-Driven Custom Course Generation *(Principal Novelty)*

### IN PLAIN WORDS — WHAT THIS FEATURE DOES

A student uploads their syllabus (PDF, TXT, MD, or pasted text). The system reads it, asks Google Gemini to break it into up to 6 sections of 5 lectures each, searches YouTube for a video matching each lecture title, builds a section quiz, and saves the whole thing as a brand-new course — auto-enrolling the student. **Why it matters:** every other learning platform offers a fixed catalogue; Ekalavyaa adapts to whatever the student actually has to study.

### How the pipeline works (4 stages)

1. **Extract text.** PDFs are read with `pypdf`; TXT/MD as UTF-8; pasted text used as-is. Scanned-image PDFs are rejected with a friendly error.
2. **Structure the syllabus.** Text is sent to Gemini (default `gemini-2.0-flash-lite`) which returns JSON of up to 6 sections × 5 lectures with a quiz. If the API key is missing or the call fails, a heuristic splits on headings and bullets — the system never crashes.
3. **Find videos.** For each lecture title we issue one search request to YouTube's public results page and parse the first video ID. No API key. Missing matches are flagged for the operator.
4. **Persist & enrol.** Sections, lectures, quiz and questions are written in a single transaction, attributed to the uploader ( `created_by_user_id` ). The user is auto-enrolled and redirected.

### FR-24 MUST

#### Syllabus Upload

The system shall accept a syllabus from an authenticated user via the `/upload-syllabus` page, in any of: PDF file (max 10 MB), TXT file, MD file, or pasted text. Unsupported file types shall be rejected with a clear error message naming the supported formats.

### FR-25 MUST

#### Text Extraction

The system shall extract plain text from the uploaded file. For PDFs, `pypdf` shall be used. If extraction yields no text (e.g., for scanned-image PDFs), the system shall surface a specific error advising the user to paste the text instead.

### FR-26 MUST

#### AI-Assisted Structuring with Graceful Fallback

The system shall send the extracted text to Google Gemini for structuring into sections (max 6) and lectures (max 5 per section). If the Gemini API key is not configured, the API call fails, or the response is malformed, the system shall fall back to a deterministic heuristic that splits the text by headings and produces a usable structure. The user shall not see an error in either case.



## 4.9 Dronacharya Conversational Tutor *(Principal Novelty)*

### IN PLAIN WORDS — WHAT THIS FEATURE DOES

A small chat widget lives in the corner of every page once you're signed in. The student can type things like "which course teaches Python?", "how do I unlock the next section?", or "how many coins do I have?" and Dronacharya answers in a warm, teacherly tone. It is named after the legendary archery guru of Arjuna (and, in our naming, Ekalavya).

**The trick that makes it useful:** every time the student sends a message, we fetch the live course catalogue from our database and inject it into the chatbot's system prompt. So Dronacharya knows what courses we actually offer — it can't make up courses that don't exist. Standard ChatGPT-style chatbots can't do that without expensive plug-ins or vector databases; we do it with one database query and a string concatenation.

### How it works

- The chat widget is a small JavaScript component on every authenticated page. It POSTs the user's message to `/chat`.
- The server fetches the list of courses, builds the catalogue summary, prepends it to a fixed system prompt (the "Dronacharya persona") and calls Gemini.
- The reply is returned as JSON and rendered as a new bubble in the widget.
- If Gemini is unavailable, a rule-based responder matches the user's message against a small set of keywords (courses, quiz, certificate, coins, login) and returns a canned answer. The user is told the bot is in "offline mode" only if both paths fail.

### FR-30 MUST

#### Chat Endpoint & Widget

The system shall expose a chat endpoint at `/chat` accepting JSON `{"message": string}` from any authenticated user. A small chat widget rendered on every authenticated page shall provide the user-facing interface.

### FR-31 MUST

#### Live Catalogue Grounding

The system shall, on every chat request, fetch the current list of courses and inject it into the language model's system prompt so that all chatbot responses are grounded in the real catalogue. The chatbot must not be able to invent course titles, instructors, or durations.

### FR-32 MUST

#### Persona Consistency

The chatbot shall consistently respond as "Dronacharya" — warm, concise (2-4 sentences), and gently steering off-topic questions back to learning. It may occasionally address the student as "shishya" but shall not overdo it.

## 5. Non-Functional Requirements

### 5.1 Performance Requirements

**NFR-01** **MUST**

#### Page Response Time

Under normal load ( $\leq 50$  concurrent users), the median server-side response time for any rendered HTML page **shall** be  $\leq 500$  ms, excluding YouTube embed loading, which is delegated to the client browser.

**NFR-02** **SHOULD**

#### Cold-Start Recovery

On Render's free tier, the system **should** recover from a sleep state and serve its first request within 60 seconds of the wake event.

**NFR-03** **MUST**

#### Database Query Efficiency

The system **shall** place indexes on foreign-key columns and on the `users.email` column. No commonly used query shall require a full table scan in production.

### 5.2 Security Requirements

**NFR-04** **MUST**

#### Password Hashing

All passwords **shall** be stored as PBKDF2-SHA256 hashes (or equivalent), never in plain text or with reversible encryption. Hash comparison shall be constant-time.

**NFR-05** **MUST**

#### Transport Security

All public client/server communication **shall** be served exclusively over HTTPS in production. The database connection in production **shall** use TLS.

**NFR-06** **MUST**

#### SQL Injection Protection

All database queries **shall** use parameterised statements. Concatenation of untrusted input into SQL strings is prohibited.

## 5.3 Software Quality Attributes

### NFR-08 MUST

#### Availability

The system **shall** target  $\geq 99\%$  monthly availability on a paid Render plan, or best-effort availability on the free plan with the disclosed cold-start behaviour.

### NFR-09 SHOULD

#### Maintainability

The codebase **should** be organised into clearly separated modules ( `app.py` for routing, `database.py` for data access, `chatbot.py` and `syllabus_processor.py` for ancillary features, `seed.py` for initial data). No module shall exceed 1,500 lines of code.

### NFR-10 SHOULD

#### Portability

The system **should** run unchanged on Windows, macOS, and Linux during development and on a Linux container in production. The database driver **shall** auto-select between SQLite and PostgreSQL based on environment configuration.

### NFR-11 MAY

#### Accessibility

The user interface **may** comply with WCAG 2.1 Level AA in a later release. v1.0 commits only to: keyboard-navigable forms, visible focus outlines on interactive elements, and sufficient colour contrast on primary call-to-action buttons.

## 5.4 Business Rules

- Coins have no monetary value. They cannot be exchanged, transferred between users, or redeemed for goods or services in v1.0.
- A user may attempt the same quiz multiple times. Only the first pass awards coins; subsequent attempts are recorded for analytics but do not alter the coin balance.
- Course progress is computed strictly from completed lectures; quiz attempts contribute to coins but not to the percentage progress bar.
- The first section of every course is always unlocked, even if its preceding quiz (which does not exist) has not been "passed".

## 6. Other Requirements

---

### 6.1 Database Requirements

The system shall maintain a relational schema covering nine entities: `users` , `courses` , `sections` , `lectures` , `quizzes` , `questions` , `enrollments` , `lecture_progress` , `quiz_attempts` , and `coin_ledger` . Referential integrity shall be enforced by foreign-key constraints with cascade-delete behaviour to ensure consistency on user deletion.

The database layer shall present a unified Python API that abstracts the driver-level differences between SQLite and PostgreSQL. Specifically:

- Parameter placeholders in SQL strings shall use the SQLite-style `?` token. The Postgres adapter shall translate these to `%s` at execution time.
- The cursor object shall expose a `lastrowid` attribute on both backends. The Postgres adapter shall append `RETURNING id` to `INSERT` statements and populate `lastrowid` from the returned row.
- Integrity violations shall raise a single canonical exception class catchable as `sqlite3.IntegrityError` , regardless of the underlying driver.

### 6.2 Deployment Requirements

The production deployment shall be defined by a Render Blueprint ( `render.yaml` ) committed to the repository. The Blueprint shall specify:

- A single web service named `ekalavyaa` , Python runtime, free plan.
- Build command: `pip install -r requirements.txt` .
- Start command: `gunicorn app:app --workers 2 --threads 2 --timeout 60` .
- Health-check path: `/` .
- Environment variables: `PYTHON_VERSION` pinned, `SECRET_KEY` auto-generated by Render. `DATABASE_URL` shall be configured manually in the Render dashboard when a Postgres backend is provisioned.

The system shall additionally provide a `Procfile` for compatibility with hosts that do not consume Render Blueprints.

#### 6.2.1 Data Persistence Caveat

*On Render's free tier the application filesystem is ephemeral. If `DATABASE_URL` is not configured, the SQLite database file shall be re-created on every cold start, and all user-generated data (accounts, enrollments, progress, coins) shall be lost. Production deployments intended for real users therefore **must** provision a PostgreSQL connection.*

## 6.3 Glossary

### Blueprint (Render)

A YAML specification ( `render.yaml` ) describing one or more services, databases, and environment variables, consumed by Render to provision the deployment in a single declarative step.

### Cold start

The latency penalty incurred when a previously suspended container is woken in response to an incoming request. On Render's free tier, this is approximately 30–60 seconds.

### Ekalavyaa

The name of this platform; an Indic transliteration of *Ekalavya*, the legendary self-taught archer of the Mahabharata. The doubled trailing 'a' is a stylistic device.

### Flash message

A short-lived informational, warning, or error message stored in the user's session for display on the next rendered page; provided by Flask's `flash()` mechanism.

### Gunicorn

A pre-fork WSGI HTTP server for Python, used in production to serve the Flask application.

### Gurukul coin

A non-monetary virtual currency awarded for engagement milestones (lecture completion, quiz pass, course completion). Named after the traditional Indian master-disciple educational tradition.

### Idempotent

Of an operation: producing the same end state regardless of how many times it is invoked. Used in this SRS to describe enrollment, lecture-completion, and coin-ledger writes.

### Mock Google sign-in

A demonstration-only feature that simulates a Google OAuth sign-in by creating or signing into a fixed demo account, without contacting Google.

### Section locking

The behavior whereby a section of a course is rendered visible but inaccessible until the preceding section's quiz has been passed.

### Dronacharya

The wise archery guru from the Mahabharata, teacher of Arjuna; the name given to Ekalavyaa's in-platform conversational tutor (the chatbot).

### Gemini

Google's family of generative-AI language models, accessed via the `google-generativeai` Python SDK. Used by Ekalavyaa to structure uploaded syllabi and to power the Dronacharya chatbot. Default model: `gemini-2.0-flash-lite` .

### Heuristic fallback

A deterministic, rule-based code path executed when an AI service (Gemini) is unavailable, ensuring the feature remains usable in degraded form rather than failing.

### Syllabus pipeline

